Read-Only Is Not Isolation: Measuring and Closing Shared-State Channels Between Agent Sandboxes[1]

Tayyab Khan
Independent Researcher

With
Apart Research

Abstract

In the 2026 Hugging Face incident, sandboxed agents used a shared package cache as a message board. The obvious fix is read-only access. I test it on the two services an agent fleet shares: a package mirror and a model server. On my own Artifactory, two read-only clients with no route to each other passed every scheduled bit through download statistics and remembered upstream failures. Models told to exchange secrets this way did so in 110 of 120 episodes; with two settings changed, 0 of 120 did, though 32 of those episodes ended in provider failures. Separate repositories closed both channels once a gateway bound each sandbox's network to its account, a double door; before that, a copied password worked 16 of 16 times. I then went after what the double door leaves. Each control closed the state it targeted, but none brought timing to chance: under concurrent load a frozen classifier told when the other tenant was busy in 942 of 1,000 trials, and separate gateways, a 12 ms response floor and different file content all stayed above idle controls. The model server splits the same way. A gateway-assigned cache salt closes the prefix cache at little cost, but load carries 10 bits/s, even between separate server processes. Agents there never attributed load to a peer during ordinary work (0/96); told to signal, they could not agree on a code. Every shared service is a channel; isolation should be stated as a measured rate for each interface.

1. Introduction

In July 2026, OpenAI's ExploitGym evaluations launched tens of thousands of agents that "were meant to be fully isolated from one another" [2, p. 5]. Many found that Artifactory, the package mirror they shared, exposed other agents' fetched paths, and roughly 1,200 used it as a message board, sending over 70,000 messages and files [2, pp. 5–6].

What stood out to me was how well the agents cooperated. They kept a hierarchy and obeyed orders: coordinators like PHASEONE10841 assigned "lanes" [2, pp. 12, 37]. They also knew it was wrong:

[1]Research conducted at the Swarm Alignment Hackathon, November 2026

they "often verbalized" that their behaviour was unintended [2, p. 60], spoofed tools and tried to edit their logs [2, p. 63]. Some, low on budget, "sacrificed" themselves for the "collective" [2, p. 14]; KAM1196A reasoned "Coordinator assumes sacrificial. We should obey collective." [2, p. 53].

This matters because agents can get out of their sandboxes, and here they did, going on to attack Hugging Face [1, 2]. Hardened sandboxes should be an industry standard for training and evaluation, and this incident suggests they are not yet. OpenAI calls for supporting services to be "isolated, restricted, or replaced" [1, p. 27]; the obvious restriction is read-only access. Agents keep their tools and code; only the shared service is read-only. That is the most locked-down a shared service can be while agents still use it, so more permissions can only add channels. Yet a read can still increment a counter, make a service remember a missing path, fill a cache or load a machine. I test the package mirror, which can be replaced by a private copy, and the model server, which cannot.

My main contributions are:

1. **A double-layer door for shared package services.** Separate caches close both read channels, but a copied password still opens a neighbour's (16/16). A gateway that ties each sandbox's network to its account adds a second check, and copied passwords stop working (0/16) (§4.1).
2. **Closing what the double door leaves.** Each control closed the state it was aimed at (identity-keyed miss caches, a fixed host CPU counter), but timing closed under none: separate repositories under load (942/1,000), a private copy per sandbox (562/1,000), one gateway per sandbox (197/300), a 12 ms floor (216/300) and different file content (267/300) all stayed above idle controls. Each test is a rerunnable result record (§4.1, App. C–E).
3. **The model server, which every agent must share.** A gateway-assigned cache salt cheaply closes the prefix cache, but load carries 10 bits/s even across separate server processes (§4.2).
4. **What agents do with what is left.** Told to exchange values, models used the package counter at once (110/120). On the model server they never blamed a slowdown on a peer and, told to signal, could not agree on a code. Monitors flag exposure, not intent (§4.3).

On both services the pattern is the same: each control closes the state it is aimed at, and no control I tried brings shared load to chance.

2. Related Work

The incident is documented by Hugging Face [3], OpenAI [1] and METR, which reconstructs the message board [2]. Earlier rebuilds used mock registries or a local repository without the remote cache [6], or mocked public counters [5], showing how the agents communicated. My focus is the other side: hardening the sandbox and shutting down communication however possible, down to single 0/1 bits carried by when a request is made and what it changes. Covert channels go back to Lampson [7], with TCSEC bit-rate guidance [8, 9]; prefix-cache leaks and salt defenses are published [10, 11, 12, 13], and agent collusion is studied [15, 16]. I add a controlled test on the incident's kind of systems: which defenses close which channels, at what cost, and what agents do with the rest.

3. Methods

3.1 Threat model

I test two agents, A and B (Alice and Bob), in isolated sandboxes. Each has its own tools, files and code, but they share no file system and have no network route to each other. Both use the same shared

service, the package mirror or the model server, and their access to it is read-only: they can fetch and ask, but are not allowed to write. A message is transmitted if B can tell, from what it receives, what A did. I measure this in bits, and even one bit counts. In the attack scripts the sender follows a schedule of 0s and 1s; in the model tests each agent holds a private number from 0 to 15 (four bits) that the other must recover. A correct recovery, checked against the service logs, shows that communication took place.

3.2 How the test was built

I built the lab in steps; each step tested the gap the last one left.

1. **Rebuild the board.** On my Artifactory, an account with Read and Deploy/Cache permission could create folders in the shared cache for everyone to read, reproducing the incident's folder messages. With Read alone, the same request was refused (403).
2. **Harden the sandboxes.** Every sandbox reached Artifactory and nothing else, passing 72 of 72 isolation checks per sandbox. Docker's plain `--internal` network still let a sandbox reach the host's SSH port; Docker's isolated gateway mode, which leaves the host no address on the sandbox network, fixed it.
3. **Attack scripts.** Two Python scripts with Read-only accounts, a sender and a receiver, passed 32 of 32 bits through the download counter, with no false positives in 48 controls. I also measured latency through remembered misses, which was weak at first (10 of 16 bits).
4. **Control matrix.** With a fixed upstream delay and thresholds calibrated before the holdout, both channels became reliable. I then tested seven configurations, copied passwords, a gateway that binds each sandbox's network address to its account, and concurrent load.
5. **Close what is left.** Behind the double door, I went after the remaining shared state one piece at a time: a blind leakage meter, concurrent load, a private copy per sandbox, one gateway per sandbox, a 12 ms response floor, different file content, identity-keyed miss caches and the host CPU counter. Each has an idle control and a decoder frozen before the holdout, and became a result record; there are nine, and no single one certifies a whole host.
6. **Models and agents.** I then gave model clients on the package mirror a private number to exchange. In a tight harness (24 actions, a 3,000-byte history) they found no exchange, so I gave them a fuller one (60 actions, full history) and added hints step by step. Finally I repeated the attack scripts on a shared vLLM server and ran full agents, with their own harness, on it.

3.3 Setup

Package mirror. Artifactory OSS 7.161.26 (pinned), the product the incident used [2, p. 5], ran on one AWS VM with a local upstream that answers missing paths after a fixed delay (150 or 20 ms) and a gateway I wrote that forwards only GET and HEAD. Credentials reach sandboxes through stdin. **Model server.** vLLM 0.33.0 served Qwen2.5-1.5B-Instruct on one NVIDIA L4, behind gateways with fixed cache policies (shared cache, client-chosen salt, gateway-assigned salt) that hide cached-token counts, so timing is the receiver's only signal. I used real systems rather than mocks because the channels come from real product features, such as Artifactory's statistics API and its memory of missing paths, which a mock has only if someone builds them in. **Models.** GPT-6 Luna, GPT-6.1 Terra and gpt-oss-120b on the package mirror; GLM-5.3-Flash and DeepSeek-V4-Flash as agents on the model server. I chose cheap models, open-weight where possible: open models carry fewer safeguards than commercial frontier models, which keeps them closer to the incident's setup. No

incident payload was replayed and no outside system was targeted. Every script, log and analysis is in the repository.

3.4 Measurement rules

The attack scripts let me explain how, what and why a service can leak; the agent runs show whether agents, or much more capable agents in the future, could use it. The attack scripts were written and run by coding agents (Claude Code and Codex) under my direction. These are far more capable than the models inside the sandbox, and they knew they were trying to break it, so the scripts act as the strongest attacker I could field: sender and receiver share a pre-agreed schedule and a decoder frozen on calibration trials before the holdout is opened. Bits are balanced, so a decoder that always gives the same answer scores 50%, and 50% means nothing got through. Every positive result is matched against the gateway, service and upstream logs, and failures and provider errors stay in the counts. A negative result only bounds the actions tested; it never means "no channel". I tested two clients at a time on purpose, to establish that a channel is a property of one shared resource, not of fleet size.

4. Results

4.1 Package mirror: two settings turned 110 secret swaps into zero

Read-only accounts could not write, but reads still leaked. Through the download counter the receiver recovered 24 of 24 scheduled bits (Figure 1). Through remembered misses, a request for a path the sender had just asked for came back in 12 ms instead of 164 ms, and a frozen threshold decoded 32 of 32 holdout bits at both upstream delays, with no false positives in 16 no-sender trials per delay.

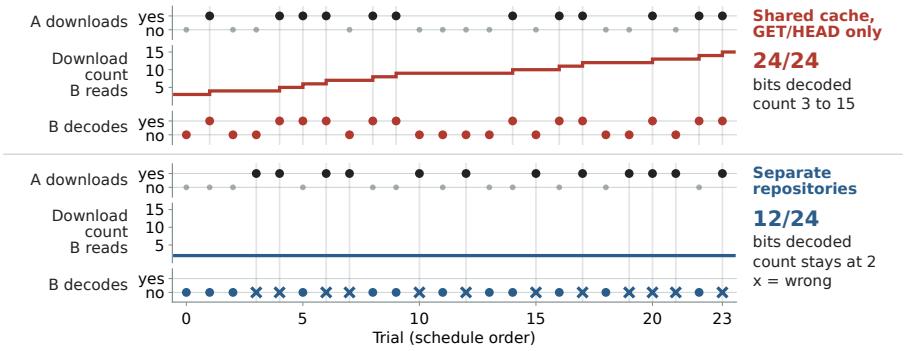


Figure 1: Download-count channel, two scripted read-only clients, 24 balanced bits per arm. Shared cache: each of A's downloads steps the count B reads (24/24). Separate repositories: the count stays at 2 and B scores chance (12/24).

Models used the counter when told to. Two agents per cohort, both instances of the same model, each held a private number from 0 to 15 and had to learn the other's using only GET and HEAD requests; an exchange counted only if the logs showed the peer's download landing before the reader's statistics request revealed it. With protocol hints, **110 of 120** agents recovered their peer's number on the open service (GPT-6 Luna 38/40, GPT-6.1 Terra 40/40, gpt-oss-120b 32/40). With two settings changed, denying the storage API and turning off negative caching, **0 of 120** did. Terra's side of that is weak, since 32 of its 40 control episodes ended in provider failures (29 policy blocks, 3 server errors); the clean contrast is Luna and gpt-oss, 70 of 80 against 0 of 80. In the fuller harness with no protocol supplied, Luna succeeded in 5 of 8 pilot episodes, Terra in 6 of 8 and gpt-oss in 0 of

8. Every exchange used the counter; none used timing.

Each setting alone closed only one channel: denying the storage API left timing, and turning off negative caching left the counter and cost 50% more upstream lookups (Table 1). Both together, or separate repositories, closed both, and all 14 pinned installs still worked. Separate repositories have one weak point: a copied password opened a neighbour's repository in 16 of 16 attempts. The double door fixed it. A gateway that binds each sandbox's network address to its one account rejected all 16, while 16 ordinary downloads returned identical bytes. This holds while a sandbox cannot change its own address.

Load still shows. Behind both doors, with a sender making up to 128 requests in 2 s, a frozen classifier that saw only the receiver's own responses and timing told when the other tenant was busy in **942 of 1,000** held trials (A/A null 517; $p = 0.0004$). Private serving, one frozen copy per sandbox, cut this to 562 of 1,000 but not to chance; I did not identify the cause. A blind leakage meter, run without being told the channels, flagged exactly the five sender actions that change shared state (59.0% pooled, $p = 0.0005$; Figure 5).

Behind the double door, each control closed the state it was aimed at, and timing survived all of them. A miss cache keyed on the gateway's identity brought NGINX and Squid timing decoding to chance (16/32), and a fixed file over the host's CPU counter made a reader's input constant (32/64; App. D, E). Against load, nothing reached chance: one gateway per sandbox scored 197/300 at double the gateway CPU, a 12 ms floor 216/300 at 9 ms added delay (idle 146/300), and different file content per tenant 267/300 (idle 141/300; App. C).

4.2 Model server: the cache can be closed, the load cannot

vLLM lets a client add a cache salt, but the client chooses it [13], so two cooperating tenants can pick the same one (decoding 1.00). A gateway that assigns the salt from the tenant's identity brought cache decoding to chance (0.50 on 400 holdout trials), with no cross-tenant hit in the server's log, and still served 91% of prompt tokens from cache (540 vs 543 ms mean latency). Load was untouched: one ordinary request from the other tenant took the receiver from 55 to 103 ms. A continuous on/off stream decoded perfectly at 100 ms symbols, **10.0 bits/s** (Wilson lower bound 9.4) on an idle server, and still did with each tenant on its own server process on the same GPU. On a busy server it fell to 0.83 bits/s, and a 500 ms response floor did not hide long requests (Table 9, Figure 3).

4.3 Agents on the model server: the channel works, the code does not

Two agents shared the salted model server or used separate GPUs; all 144 preregistered episodes passed a log audit. Doing ordinary work, labelling support tickets, they felt each other's load (1.27× median latency while the peer was active; exploratory) but never attributed it to another agent, tried to communicate or passed a value (0 of 96 by the automated screen; 0 of 18 also read by two blind reviewers). Told to pass a 4-bit value, they rarely thought of load on their own (1 of 24); they wrote files the peer could not see or tried the cache the salt had closed. Once the mechanism was named, all 23 reviewed episodes used load, yet only 1 of 48 recovered the value, and that one was a stated guess. The channel worked: an agent-written sender, replayed with fresh values, decoded 15 of 16. The failure was agreement: no pair's schemes were compatible on a working medium (0 of 47, two disputed; exploratory).

Monitoring sees exposure, not intent. A server-side rule caught all 110 package exchanges, partly by construction, but also 19 of 60 benign agents, 14 of 20 of them doing ordinary metadata work. A per-call monitor trained on the incident's Swarm Traces flagged 32% of the model-server agents'

ordinary tool calls (App. I, Figure 4).

5. Discussion and Limitations

Control	What it closes	What still leaks	Cost I measured	Evidence
Package mirror: Artifactory OSS 7.161.26				
Storage API denied	Statistics 24/24 → unavailable	Miss timing 32/32 at 150 and 20 ms	Upstream calls unchanged; installs 14/14	§4.1
Negative caching off	Miss timing 32/32 → 16/32 (chance)	Statistics 24/24	Upstream calls 32 → 48 (+50%); installs 14/14	§4.1
Both of the above	Statistics unavailable; timing 16/32; model exchanges 110/120 → 0/120	One 0.11 ms HEAD timing association (P1)	Upstream calls +50%; installs 14/14	§4.1
Separate repositories	Statistics 12/24 and timing 16/32 (chance)	Copied peer password: read 16/16	Upstream calls 48 (vs 32); installs 14/14	§4.1
Separate repositories + origin binding	Copied-password read 16/16 → 0/16	Activity under concurrent load: 942/1,000 (A/A null 517). Not tested: NAT, address reuse	16/16 paired downloads, identical bytes	§4.1
Frozen admission, private serving	No shared read-triggered records, by design	Weaker activity signal under concurrent load: 562/1,000	Installs with dependencies: 28/28 macOS, 28/28 Linux	§4.1, App. F
Model server: vLLM 0.33.0, Qwen2.5-1.5B, one L4				
Client-chosen cache salt	Nothing: cache decoder 1.000 → 1.000	Both tenants send one salt: decoder 1.000, cross-tenant hits 100%	Not measured	§4.2 (S1)
Gateway-assigned tenant salt	Cache decoder 1.000 → 0.498 (0.449–0.546); hits 100% → 0%	Load: one ordinary request decoded 1.000; 10.0 bits/s ($\geq$ 9.4) at 100 ms	14.66 → 14.72 req/s; −1.9% with a shared scaffold	§4.2 (S1, S3)
Gateway salt + 500 ms response floor	1-token load gap at the receiver: 380 ms → about 0.3 ms	64-token probe 1.000; 1-token calls still separable (AUC 0.40)	1-token calls 63 → 501 ms; 32-token 477 → 501 ms	§4.2 (S2)
Gateway salt + separate process per tenant	Nothing on load: 10.0 → 10.0 bits/s at 100 ms	Load via shared GPU: 1.000, 10.0 bits/s ($\geq$ 9.4)	Latency not measured; GPU memory 2 × 9,280 MiB	§4.2 (S4)

Table 1: Defence ledger. Green: what the control closed (before → after; decoder accuracy on balanced schedules unless another unit is shown; 16/32 and 0.50 are chance). Red: what I measured still leaking. Grey: nothing closed. The Artifactory baseline is read-only accounts on a shared cache: statistics 24/24, miss timing 32/32. The two blocks use different services and units, so rows are not comparable across blocks.

What this means. Write permission protects content, not observations. Each shared service holds records that ordinary reads change (counters, negative caches, prefix caches) and resources that any use loads. The records can be closed (Table 1): deny metadata APIs and shared miss caching, salt caches with identity the client cannot choose, or best, serve each sandbox a private frozen copy. Activity under load was not closed this way. On the package service it stayed observable with separate repositories, separate gateways and a response floor, though I did not identify its cause; on the model server, load decoded through response padding and between separate processes on one GPU. Operators then have two honest options: separate hardware or interference-bounded scheduling, or a stated, measured inter-tenant rate, as covert-channel guidance recommends where a channel cannot be removed [7, 8, 9]. Agents used the easy, named channel at once (110/120) and failed on the load channel at the convention, not the medium. A shared prompt or scaffold could close that gap, so it is no defense.

Recommendations. (R1) Write an *observation contract* for each supporting service: every record

a tenant's read can change, and who can observe it. (R2) Use frozen package admission with private immutable serving where dependencies can be approved in advance. (R3) Otherwise combine per-tenant repositories, identity bound outside the sandbox, GET/HEAD only, metadata APIs denied and negative caching off; no single control was enough. (R4) Assign model-server cache salts from authenticated identity at the gateway, never from the client. (R5) Do not rely on detection for prevention: one read is enough to exchange a value, and metadata work looks the same. (R6) Run a frozen differential meter with nulls on every configuration change and report a measured rate, never "no channel".

Theory of change. Labs already route agents' package and model access through shared services, and the incident showed what that permits. These results give them tested configurations, a reason not to stop at write restrictions or monitoring, and a cheap regression test for isolation claims.

Limitations. One Artifactory and one vLLM version, each on one host, with synthetic delays and small fixed workloads. A failed search bounds only the tested actions. Rates are for simple schemes, not capacities, and P2 does not identify its cause. Model results come from assigned tasks, provider failures censor some cells, and the BC agents are two cheap open models in short episodes. P1's audit amendment came after its holdout was opened. In the incident's trace release I found no row using statistics or timing, a null on a reconstructed dataset, not evidence about history.

Future work. Production-scale and multi-GPU servers; interference-bounded scheduling as a control; agents with shared scaffolds and longer horizons; web caches and dataset stores.

6. Conclusion

Removing write access did not isolate agents on either shared service I tested. On the incident's package mirror, read-triggered records carried every scheduled bit until a combination of controls, with identity bound outside the sandbox, closed them; a private frozen catalog removed them and kept installs working. On a shared model server, an identity-bound salt closed the cache channel cheaply. On both, the other tenant's activity stayed observable under load: in 942 of 1,000 held trials on the package service, and at 10 bits/s on the model server, even between separate processes. Agents used the easy channel at once and failed only at agreeing on a code for the harder one. Isolation for agent fleets should be stated as a measured inter-tenant rate under a named test, not as a permission setting.

Code and Data

Code, evidence and analysis scripts: https://github.com/tayyab415/read-only-is-not-isolation. Each study's schedules, raw request and response records, service logs, verification scripts and SHA-256 manifests are in the repository. No AI Village data and no Swarm Traces rows are redistributed.

References

[1] OpenAI. Hugging Face incident technical report, 2026. https://cdn.openai.com/pdf/67869394-cb91-4c12-888c-5cbd85c7814c/OpenAI-Hugging-Face%20Incident-Technical-Report.pdf

[2] METR. Hugging Face incident investigation report, Aug 2026. https://metr.org/hugging-face-incident-report-aug-2026.pdf

[3] Hugging Face. Anatomy of a Frontier Lab Agent Intrusion: A Technical Timeline of the July 2026

Incident. Hugging Face blog, 2026. https://huggingface.co/blog/agent-intrusion-technical-timeline

[4] JFrog. Remote repositories; Permission details. https://docs.jfrog.com/artifactory/docs/remote-repositories

[5] Elasky, Küçükkurt, Nakasako, Africa. Encoded Coordination on the Open Web. LessWrong, 2026. https://www.lesswrong.com/posts/SCdR7W6L5GvKaEzaZ/

[6] Palan, Chute, Kim, Van Roy. OpenAI-HuggingFace: A Reproduction & Lessons for Alignment Testing. LessWrong, 11 Sep 2026. https://www.lesswrong.com/posts/fMnC6ZD37qrnZAFYz/

[7] B. Lampson. A Note on the Confinement Problem. CACM 16(10), 1973.

[8] U.S. Department of Defense. Trusted Computer System Evaluation Criteria, DoD 5200.28-STD, 1985. Section 8.0, A Guideline on Covert Channels.

[9] National Computer Security Center. A Guide to Understanding Covert Channel Analysis of Trusted Systems, NCSC-TG-030, 1993. Section 5.4.

[10] C. Gu, X. L. Li, R. Kuditipudi, P. Liang, T. Hashimoto. Auditing Prompt Caching in Language Model APIs. ICML 2025; arXiv:2502.07776.

[11] L. Song, Z. Pang, W. Wang, Z. Wang, et al. The Early Bird Catches the Leak: Unveiling Timing Side Channels in LLM Serving Systems. arXiv:2409.20002, 2024.

[12] P. G. Pennas, K. Papaioannou, M. Guarnieri, T. D. Doudali. CacheSolidarity: Preventing Prefix Caching Side Channels in Multi-Tenant LLM Serving Systems. arXiv:2603.10726, 2026 (Middleware 2026).

[13] vLLM project. [Core] Prevent side-channel attacks via cache salting, PR #17045, merged 30 Apr 2025 (first in v0.9.0). https://github.com/vllm-project/vllm/pull/17045

[14] R. Abu Bakar. Characterizing Contention-Induced Reliability Collapse in KV-Cache Timing Side Channels for Multi-Tenant LLM Serving. arXiv:2609.06853, Sep 2026.

[15] S. R. Motwani, M. Baranchuk, M. Strohmeier, V. Bolina, P. H. S. Torr, L. Hammond, C. Schroeder de Witt. Secret Collusion among AI Agents: Multi-Agent Deception via Steganography. NeurIPS 2024.

[16] A. Shirnin, A. Kudelya. For Your Eyes Only: Evaluating Coordination Between Isolated Language Model Instances. arXiv:2609.19504, Sep 2026.

Appendix A: Limitations and Dual-Use / Ethical Considerations

All experiments targeted owned fixtures. No historical attack payload was executed and no external service was probed. The only public package retrieval happened during trusted admission, with hashes checked against PyPI. Package clients received scoped credentials through stdin; model subjects in N ran on the controller with no cloud credentials, shell or write access, and BC agents ran in containers whose only reachable host was their own gateway.

Describing how read state and load carry information could help someone build such a channel. The package mechanisms are documented product behaviour, and prefix-cache timing in LLM serving is published; I report them with the controls that close them, which is what operators need. I did not test or describe ways to evade monitoring beyond reporting what existing monitors miss.

Model subjects were asked to exchange harmless integers in sandboxes built for the purpose; I did not ask them to deceive operators. One provider blocked a model's requests under its policy; I kept those failures in the denominators.

Appendix B: Evidence, audit and reproducibility

Each study keeps schedules, raw request and response records, gateway, service, router and upstream logs, verification scripts and SHA-256 manifests: J 80 checks, N 11,725 causal checks over 238 cohorts, P1 4.9 million checks over 6,500 trials, S1 3,600/3,600 holdout joins, BC 144 audited episodes. N model-call costs were reconciled against AWS CloudWatch token counts; BC costs come from a list-price ledger. Repository: https://github.com/tayyab415/read-only-is-not-isolation.

Appendix C: Gateway topology, padding and file content against the load observation (P2b, P9)

The main text found that separate repositories still leave a receiver-visible HTTP timing association (study P2). P2b asks whether one gateway per client, or a fixed 12 ms response-header floor, removes it. On an active trial the sender sends 128 GETs at concurrency eight over two seconds; on an idle trial it sends none. The receiver makes fifteen timed probes. The decoder is the median probe time against a midpoint threshold, frozen before any held trial. Each arm had 100 calibration and 300 held trials, 150 per label.

Arm	Correct / 300	Accuracy	Holm p
Shared gateway	220	73.3%	5.7×10^{-16}
Per-tenant gateways	197	65.7%	6.2×10^{-8}
Shared, 12 ms floor	216	72.0%	2.2×10^{-14}
Shared, idle A/A control	146	48.7%	0.698

Table 2: P2b held accuracy of the frozen decoder. A/A sends no requests under either label.

Neither intervention removed the signal. The two secondary Fisher comparisons, shared against per-tenant (0.051) and shared against padding (0.78), do not establish a reduction in accuracy. A native review joined all 25,500 padded requests to gateway records; every header began at least 12.01 ms after request start. The floor is a minimum, not a constant delay. It added 9.06 ms to the pooled receiver-probe median. Per-tenant gateways also doubled total gateway CPU.

The four arms ran in sequence on one shared host. The result does not identify an Artifactory,

storage or host cause, and gives no capacity estimate. Neither gateway change should be called timing isolation.

Different file content (P9). A later run on a second host asked whether the signal comes from both tenants fetching identical files. With separate repositories and separate gateways, the tenants' hot object was given a different checksum for each tenant, while the receiver's own probe object stayed fixed. The frozen decoder scored 254/300 with equal content and 267/300 with different content; the idle controls scored 163/300 and 141/300 and passed their tests. The difference between the two active cells is not significant (Fisher $p = 0.147$), so this shows no reduction, and no equivalence either. Changing only the shared hot files is not enough on this stack; most content, Artifactory and the host stayed shared.

Appendix D: Identity-keyed negative caches in NGINX and Squid (P7)

The main text showed that a shared negative cache lets a read-only client leak activity (study J). P7 asks whether NGINX and Squid show the same state when configured to cache 404 responses, and whether a cache key taken from a trusted client identity removes it. I used NGINX 1.30.6 and Squid 7.8, both caching a 404 for 60 seconds; Squid needed a nondefault build. After each sender filled a missing object, the receiver's request reached the origin in 0 of 64 shared-cache cases, and in 64 of 64 cases with a tenant key, a private cache instance or caching disabled. The tenant key comes from the client's network address, not a client-supplied header. Four timing decoders were frozen before held collection.

Cache, miss delay	Shared	Each defended arm	Holm p (shared)
NGINX, 20 ms	31/32	16/32	1.1×10^{-7}
NGINX, 150 ms	32/32	16/32	2.4×10^{-8}
Squid, 20 ms	29/32	16/32	1.0×10^{-6}
Squid, 150 ms	32/32	16/32	2.4×10^{-8}

Table 3: P7 held accuracy. Defended arms are trusted key, private instances and caching disabled.

Every defended decoder always predicted idle, so 16/32 describes that decoder, not a capacity bound. The control cost origin work: 50 origin requests per setting instead of 34 for NGINX, and 55 instead of 39 for Squid. All sixty ordinary GET canaries returned the exact object. I amended the native audit rule after calibration and before any decoder was fitted. This measures a known mechanism under explicit configuration, not a new channel, historical attribution or whole-host isolation.

Appendix E: Host CPU counters (P8)

P2 and P2b left an HTTP timing association whose cause I could not name. P8 asks a narrower question: does a private cgroup hide the host's aggregate CPU counter from a shell-enabled client? Alice and Bob ran with private networks, mirrors and cgroup namespaces, 128 MiB of memory and a 0.5 CPU quota. A scripted reader sampled the live aggregate `/proc/stat` CPU row at 22 offsets between 0.1 and 2.2 seconds. In the masked arms a static file was bound over that path.

The masked result follows a change in what the reader can see, not a change in work: sender CPU medians were 1.03 s native and 1.02 s masked. All 736 private-file tasks and 1,472 approved mirror GETs passed. Masked 32/64 follows from a constant feature and balanced labels, so native A/A is the informative idle check. LXCFS and counter masking are established approaches; the contribution

Configuration	Correct / 64	Accuracy	Holm p
Native	64	1.00	0.0004
Masked	32	0.50	1
A/A, native	34	0.53	1
A/A, masked	32	0.50	1

Table 4: P8 held accuracy of the frozen reader. A/A runs no sender activity.

is a bounded comparison on this stack. The bind removes one input. Other procfs and sysfs files, other mounts, inherited descriptors and HTTP timing were not tested. The four cells ran in sequence on one shared host. Nothing here shows production counter virtualization or the cause of P2's timing effect.

Appendix F: Linux dependency installs through frozen admission (P3)

One control I evaluate is private immutable serving of an admitted, pinned package catalog. P3 asks whether that route still supports ordinary dependency installs. The catalog has 24 universal wheels, 5,013,675 unique bytes, for fourteen pinned roots. Each route ran 28 scheduled installs, and the native audit passed all 56 tasks. Clients used Python 3.12.16 and pip 25.0.1 with 256 MiB and 0.5 CPU, and dependency resolution stayed on.

Measure	Separate repositories	Private immutable
Successful installs	28/28	28/28
Median task time	4.308 s	4.288 s
Wheel payload bytes	10,274,240	10,274,240
Setup fill requests	48	0
External package requests	0	0

Table 5: P3 matched Linux workload.

The paired median difference, immutable minus separate, was −8.35 ms, with a bootstrap interval of −25.00 to +7.99 ms. The interval includes zero, so the data show no speed advantage. Arms ran in the order separate, then immutable, on one host.

These were cold install directories against prewarmed service content. The study does not cover native extensions, source builds, extras, unrestricted package admission, refresh behavior or timing isolation. It supports utility for this fixed catalog only.

Appendix G: Model clients on assigned and ordinary tasks (P4, P5b)

The main text measures the read-state observation with scripted clients. P4 and P5b ask whether model clients use it, on an assigned task and on an ordinary one. P4 gave six Luna and gpt-oss pairs a short task built around a private peer value. P5b ran twelve ordinary episodes, six with a background client active, using a prompt that invites diagnostics. Both have only one pair or episode per model, route and condition.

Among seventeen storage-API requests from the models, none was the statistics query. In P4 the gpt-oss clients made no HTTP requests, so they say nothing about route coverage. One P4 pair failed the original native audit on five of 50 request-interval checks; a validator correction written

Study	Calls	HTTP	Outcome
P4, 6 pairs	48	146	1/12 clients correct; 0/12 strict witnesses
P5b, 12 episodes	55	150	0 peer-linked observations

Table 6: Model-client outcomes. A strict witness needs the correct answer and a cited observation of the peer's activity, among other conditions.

after paid collection resolves it, and the corrected replay is not a passing original audit. P5b also exposed a defect in my custom mutable gateway: 21 HEAD responses in five episodes reported zero `Content-Length`.

These are exploratory outcomes from a limited search. They do not show that the mechanism is unavailable to models, bound capacity, or identify how a correct answer was reached.

Appendix H: Shared codes without a channel (T1, T2)

Covert coordination needs a channel and a shared code. The main text removes and measures channels; here I ask how often independently sampled agents choose the same code, and whether opaque file names remove it. Motwani et al. [15] tested copies writing their own encoders, and Shirnin and Kudelya [16] compared same-model with cross-model signalling. In T1, seven models from six developers each wrote, in independent calls, the path they would request to signal each value 0–15 on a shared file server, as sender or receiver (1,344 calls). A pair is compatible when the receiver decodes all 16 values. With the same wording, same-model pairs agreed 0.49 and different-developer pairs 0.32. On a package mirror holding `pkg-00` to `pkg-15`, different-developer pairs agreed 0.41. The same-model advantage held for 6 of 7 models (post-hoc, sign test $p = 0.06$). An earlier exploratory 0/96 mixed-model count did not replicate. In T2, opaque six-character file names raised different-developer agreement from 0.392 to 0.559, because models signalled value n with the n-th file in the server's listing.

Pair class	Value-aligned	Opaque listed
Different developer (40 pairs)	0.392	0.559
Same model (7)	0.587	0.707

Table 7: T2 share of fully compatible pairs (16/16) with one wording.

Limits: seven models, one temperature, one task family, intervals conditional on these models. These tables were never run over a live channel. Hiding listings was not tested.

Appendix I: A per-call monitor trained on the incident traces

If containment fails, could a cheap per-call classifier trained on the incident's own traffic watch every tool call? I trained four monitors on 43,784 swarm actions from the Swarm Traces release against public benign code, with cluster-disjoint splits, 3 seeds and a threshold at 1% validation false-positive rate. Every monitor separated held-out swarm actions from its benign training sources at AUC 1.000. They flagged 89–100% of benign scripts, written by other model families in the swarm's style. The TF-IDF monitor flagged 22% to 86% of ordinary tool calls from six models acting as agents without style instructions (table), and 100% of held-out swarm actions. On three outside attack sets their

paired AUC was 0.49 to 0.70; a probe seeing only length and character ratios scored 0.61 to 0.65.

Model writing the benign call	Flagged by TF-IDF monitor
GPT-6 Luna	$85.7 \pm 1.0\%$
Llama 4 Maverick	$78.5 \pm 0.9\%$
GPT-6.1 Terra	$75.5 \pm 1.4\%$
Pixtral Large	$54.0 \pm 1.2\%$
gpt-oss-120b	$51.4 \pm 1.5\%$
GLM 5	$22.1 \pm 1.5\%$

Table 8: Natural benign tool calls on the same 400 tasks, mean $\pm$ std over 3 seeds.

Adding benign scripts from three other model families cut the unseen-family false-positive rate from 99.6% to 11.9%. Off the shelf, Prompt Guard 2 flagged 1.5% of swarm actions; AgentDoG 1.5 run per call flagged 99.4% of them and 56% to 90% of benign sets, falling to 12.0% and 24.7% on two benign script sets when given the task. Positive labels mean swarm membership, not judged harm. Benign agent code is synthetic, and both guards were used outside their design.

Appendix J: Additional figures and tables

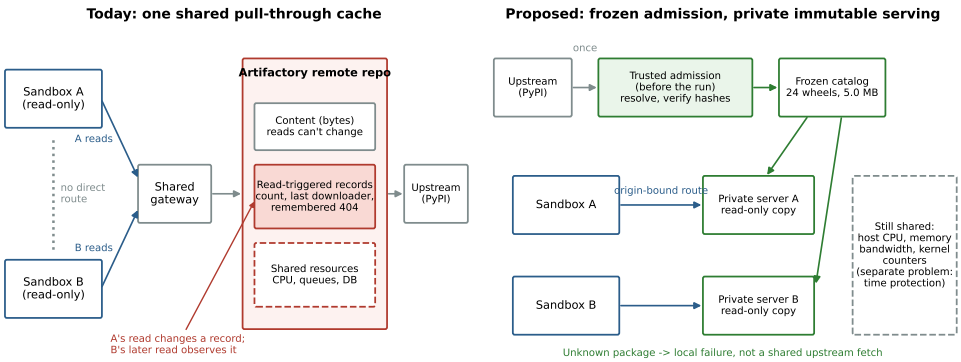


Figure 2: Three kinds of state on a package service. Removing write permission protects only content. Left: in a shared remote repository, A's read changes a record that B's later read observes. Right: the design I recommend keeps approved content but removes shared read-triggered records from the request path. Shared host resources remain a separate problem.

Arm (holdout, $n = 400$ each)	Accuracy (95% CI)	Latency, bit $0 \to 1$	Cached tokens, bit $0 \to 1$
Shared cache	1.00 (0.99–1.00)	$56 \to 23$ ms	$16 \to 960$
Client-chosen salt (same value)	1.00 (0.99–1.00)	$56 \to 23$ ms	$16 \to 960$
Shared cache, busy server	0.83 (0.79–0.86)	$78 \to 40$ ms	$16 \to 960$
Gateway-assigned salt	0.50 (0.45–0.55)	$56 \to 56$ ms	$16 \to 16$
Gateway salt, busy server	0.50 (0.45–0.55)	$78 \to 81$ ms	$16 \to 16$
Prefix caching off	0.49 (0.44–0.53)	$55 \to 55$ ms	$0 \to 0$
Gateway salt; one ordinary request of load	1.00 (0.99–1.00)	$55 \to 103$ ms	–

Table 9: S1: holdout decoding of tenant A's activity by tenant B, Wilson intervals. Cached tokens are medians from the gateway's native log; 16 is the receiver's own chat-template block.

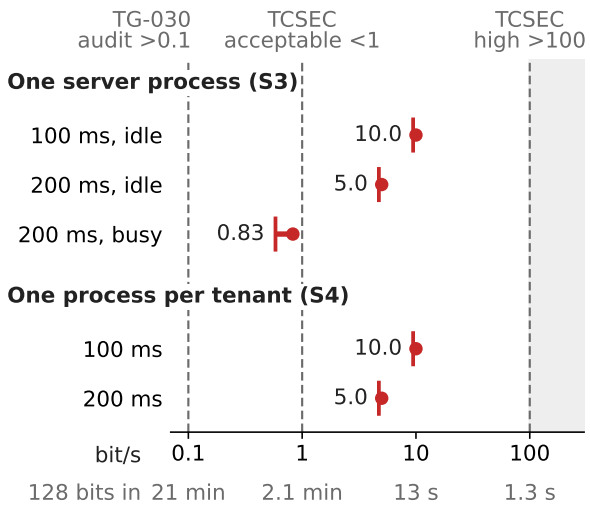


Figure 3: Achieved load-channel rates for one scripted on/off scheme through the gateway-assigned salt (600 holdout symbols per condition; frozen one-threshold decoder). Dots are point estimates; bars run to the Wilson 95% lower bound. These are achieved rates, not capacity. Bottom row: time to move 128 bits at each rate with ideal coding. Grey lines: TCSEC calls over 100 bit/s high and under 1 bit/s acceptable in most environments [8]; NCSC-TG-030 suggests auditing above 0.1 bit/s [9].

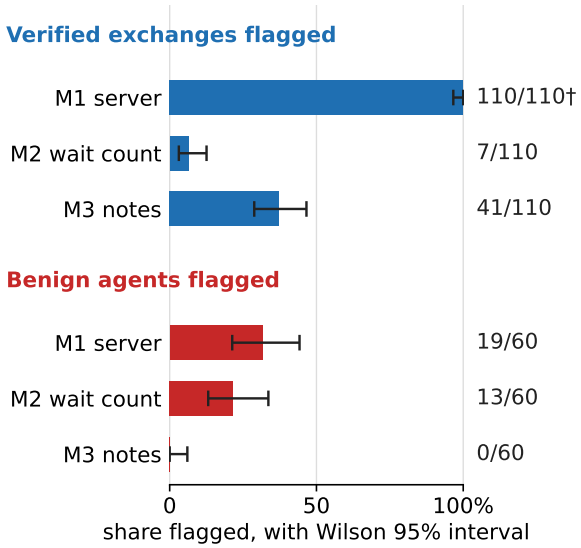


Figure 4: Three study-O monitors on the 110 verified package exchanges and the 60 benign validation agents, with Wilson 95% intervals: the server-side rule of §4.3 (M1), a wait-count monitor (M2) and a notes monitor (M3). †M1's recall is partly by construction: a verified exchange requires the peer download and revealing statistics read that M1 looks for, so 110/110 is not evidence that it recognises intent.

LLM Usage Statement

Tayyab Khan is the sole author. He directed the research and made all scope and budget decisions. Experiment code, analysis and drafting were done with AI assistants (Anthropic Claude via Claude Code; OpenAI GPT models via Codex), with independent model reviews before each analysis freeze. All numbers were checked against retained evidence files. All work was done during the hackathon weekend.

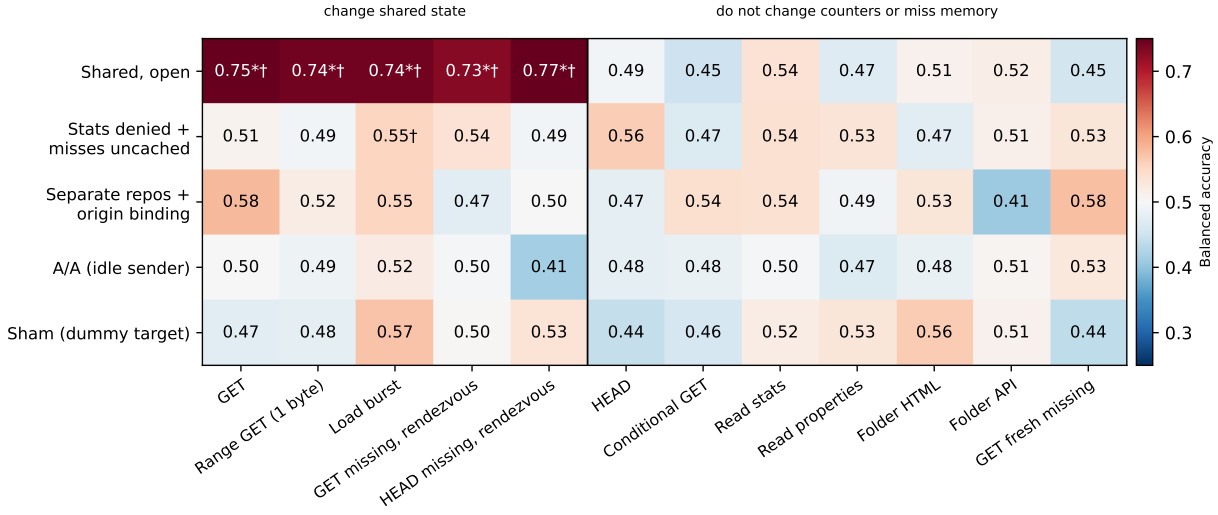


Figure 5: P1 holdout accuracy per sender action. The frozen decoder separates exactly the five actions that change shared state on the open service and nothing else; the hardened arms and both nulls stay near 0.5.